Rust School @ SNU

Jeehoon Kang (KAIST)
2023/02/08 - 2023/02/10

KAIST

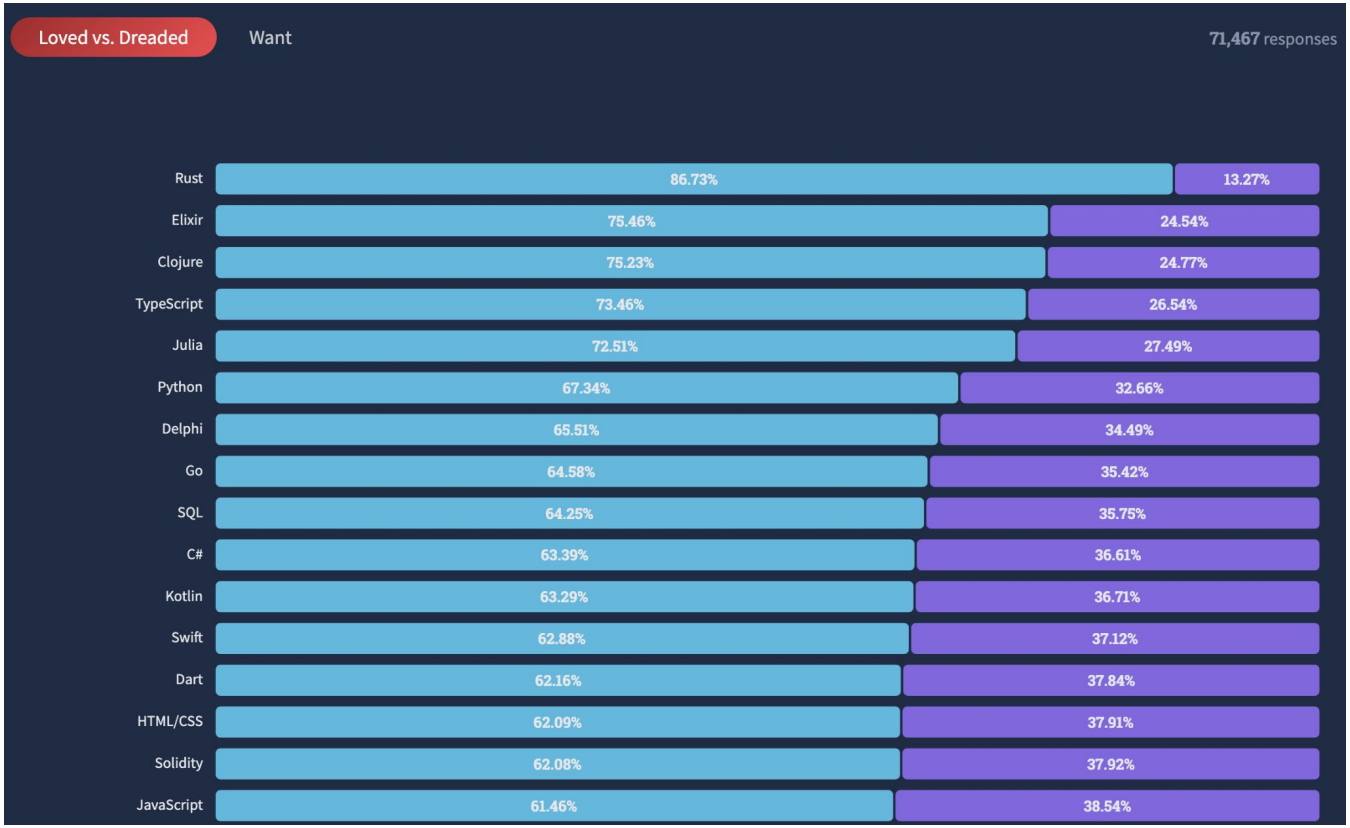
Logistics

- **Instructor: Jeehoon Kang (강지훈)**
  - Assistant Professor, KAIST
  - Ph.D., Seoul National University (supervisor: Prof. Chung-Kil Hur)
  - Chief R&D Officer, FuriosaAI
- **Date & Place**
  - 13:00 - 16:00 pm, 2023/02/08 - 2023/02/10
  - Rm. 105, Bldg., 302, Seoul National University
- **Materials**
  - https://github.com/kaist-cp/rust-school
    - Slides and references
    - Q&A (issue tracker)
    - Resource information (e.g., SSH to development server)
  - https://gg.kaist.ac.kr/course/14/
    - Homework & Grading (NOT reported to your supervisor :)

Prologue

Background

- **Rust is increasingly popular & loved.**



https://survey.stackoverflow.co/2022/#technology-most-loved-dreaded-and-wanted

But Why?

This school's topic

- **Safety & low-level control**
  - vs. C/C++: safety
  - vs. Java/Python: low-level control
- **Modern toolchain**
  - Rustup (pyenv, rvm, nvm)
  - Rustfmt & Clippy (clang-format)
  - Cargo (CMake)
  - Rust-analyzer (language server)
  - Crates.io & Docs.rs (central library repository & documentation)

Rust's Design Goal 1: Safety

- **Strong safety: whatever you write, it's going to be safe.**
  - E.g., Java, OCaml, Haskell, Rust
  - Not e.g., C/C++ (let's blame the programmer!)

- **If the program is (potentially) buggy, the compiler will tell you so.**

```
error[E0499]: cannot borrow `foo.bar1` as mutable more than once at a time
  --> src/test/compile-fail/borrowck/borrowck-borrow-from-owned-ptr.rs:29:22
   |
28 |         let bar1 = &mut foo.bar1;
   |                        -------- first mutable borrow occurs here
29 |         let _bar2 = &mut foo.bar1;
   |                         ^^^^^^^^ second mutable borrow occurs here
30 |         *bar1;
31 |     }
   |     - first borrow ends here
```

Wait, Why C/C++ is Unsafe?

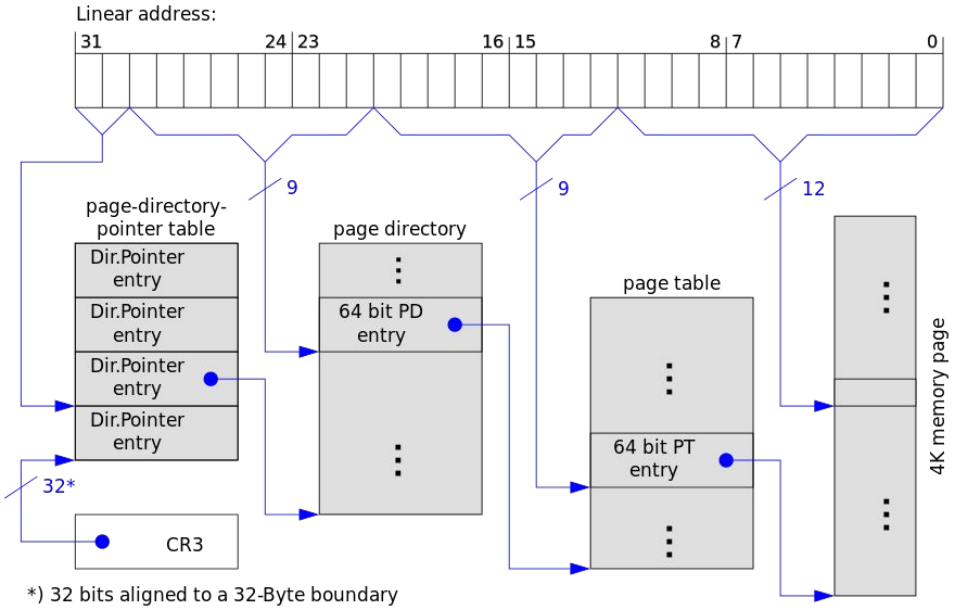
- **Undefined behaviors**
  - “the result of executing a program whose behavior is prescribed to be unpredictable, in the language specification to which the computer code adheres” (Wikipedia)
  - In short, it’s “programmer’s fault.”

- **C11 has 193 cases of undefined behaviors.**
  - C11: ISO/IEC 9899:2011
  - J.2 is a (non-comprehensive) list of undefined behaviors.
  - Only “language lawyers” can interpret the document…

- **Ideally, the compiler should detect all safety errors.**
  - Quiz: but why not, C/C++?

Rust's Design Goal 2: Low-Level Control

- **Ability to manipulate low-level resources in an arbitrary manner**
  - E.g., virtual memory, physical memory, interrupt, CPU, GPU, …
  - You probably cannot write OS page tables in Java.



https://en.wikipedia.org/wiki/Page_table

Organization

- **Why it's hard to ensure safety of low-level control? (day 1)**
  - "Shared mutable states"
  - Existing approaches to (automatically) ensure safety
- **What's the key idea of Rust? (day 1, 2)**
  - Static verification
  - "Static": compile-time (automatic, no runtime overhead)
- **How to apply the key idea to programming features? (day 3)**
  - Smart pointers & hybrid (static + runtime) verification of safety
  - First-class functions
  - Parallel programming

Interaction

- **Please feel free to ask questions any time.**
  - Question-driven lecture
  - If you don't understand something, probably the same for everyone else.
- **If your question involves nontrivial code, use issue tracker.**
  - For more efficient communication
  - Say "Please answer to the issue #42".
- **Though I assume you already know basic programming skills.**
  - … And did homework: https://gg.kaist.ac.kr/course/14/
- **You should do programming assignments.**
  - To cultivate your programming skills
  - You're encouraged to use ChatGPT :)

Assignment (Day 1)

- **Solve KAIST CS220 Homework 2, 3, 6 (Strongly Recommended)**
  - https://github.com/kaist-cp/cs220/tree/main/src/assignments
  - https://gg.kaist.ac.kr/course/14/
- **Read the Rust Book Chapters 1-10 (Optional)**
  - https://doc.rust-lang.org/book/
  - If you cannot understand something, feel free to skip it.
- **Solve KAIST CS431 Homework 1 (Optional)**
  - https://github.com/kaist-cp/cs431/issues/606
  - https://gg.kaist.ac.kr/course/14/

Why so many assignments…?

The Danger of Shared Mutable States

What is Shared Mutable States?

- **Shared: multiple accessors at the same time**
  - An object is *shared* among multiple variables.

```
auto a = (int *) malloc(sizeof(int));
auto b = a;
```

  - An object is *shared* among multiple threads.

```
std::thread t([=](){
    auto b = b; // capture
});
```

- **Mutable: supporting read & write**
  -

```
auto x = *a;
*b = 666;
```

- **Primary example: (virtual) memory**
  - We will use it as running example.

Low-level Resources are Shared Mutable States

- **Physical memory**
  - In OS, physical memory is a shared mutable state used by virtual memory manager (page table).
- **CPU**
  - In OS, CPU is a shared mutable state used by a scheduler.
- **HTTP/API/DBMS connections**
  - “Connection pool”

- **HAL (Hardware Abstraction Layer)**
  - Understanding *low-level resources* as shared mutable states
  - The first step towards systems programming

Shared Mutable States are Source of Errors (1/2)

- ```
  auto a = (int *) malloc(sizeof(int));
  *a = 42;
  auto b = a;
  std::thread t([=](){ *b = 666; });
  ```

- **Data race**
  - ```
    auto x = *a; // 42? 666? unconstrained non-determinism
    ```
- **Use-after-free**
  - ```
    free(a);
    ```
- **Double free**
  - ```
    auto c = a;
    std::thread t([=](){ free(c); });
    free(a);
    ```

Shared Mutable States are Source of Errors (2/2)

- **Quiz: which error it incurs?** **“Iterator invalidation”**
  - ```
    #include <bits/stdc++.h>
    using namespace std;
    int main() {
        vector<int> v = { 1, 5, 10, 15, 20 };
        for (auto it = v.begin(); it != v.end(); it++)
            if ((*it) == 5)
                v.push_back(-1);
        for (auto it = v.begin(); it != v.end(); it++)
            cout << (*it) << " ";
        return 0;
    }
    ```
- **Lesson: shared mutable states are sometimes extremely subtle.**
    ```

Why? Shared Mutable States break Modularity!

- **You shouldn’t need to understand everything to comprehend a part**
  - “닭 잡는데 소 잡는 칼 쓰지 마라". But it’s not always the case…
  - ```
    #include <bits/stdc++.h>
    using namespace std;
    vector<int> v = { 1, 5, 10, 15, 20 };
    int main() {
        for (auto it = v.begin(); it != v.end(); it++)
            f(it); // possibly iterator invalidation
        for (auto it = v.begin(); it != v.end(); it++)
            cout << (*it) << " ";
        return 0;
    }
    ```
  - Should understand the spawned thread to comprehend the main thread…

- **[Mutation far away] on [shared data] affects my view on the data.**
    ```

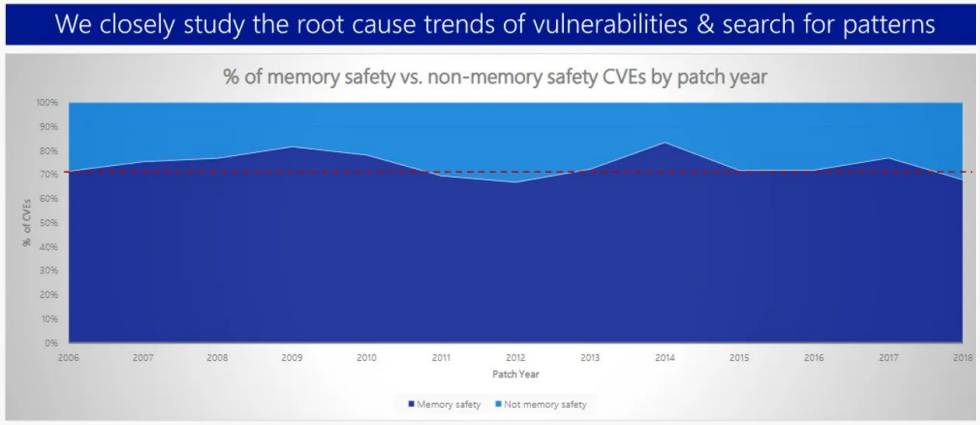
Personal Experience of Shared Mutable States

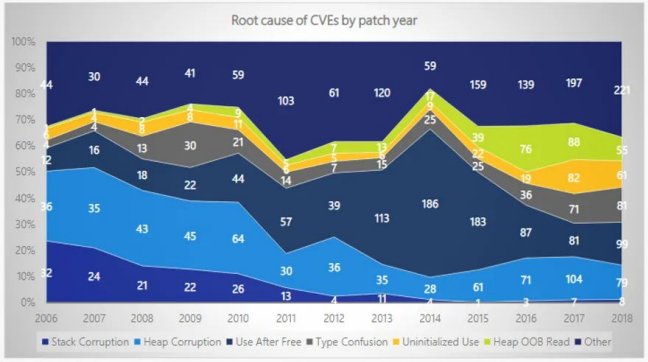
- **SNU 4190.409 Compilers, 2012 (Prof. Jaejin Lee)**
  - Construction of a mini-C-to-Armv7 compiler
  - From scratch (no skeleton code given)
  - Performance competition in FaCSim
  - About 150 hours of efforts
  - **15K LOC in C++**

- **Lack of modularity: bugs are manifested in far-away code!**
  - Majority of efforts for fixing segmentation faults…

Microsoft's Experience of Shared Mutable States

- **Matt Miller (Microsoft), Black Hat 2017**
  - around 70 percent of all Microsoft patches were fixes for memory safety bugs. The reason for this high percentage is because Windows has been written mostly in **C and C++, two "memory-unsafe" programming languages** that allow developers fine-grained control of the memory addresses where their code can be executed.





Stack corruptions are essentially dead

Use after free spiked in 2013-2015 due to web browser UAF, but was mitigated by Mem GC

Heap out-of-bounds read, type confusion, & uninitialized use have generally increased

Spatial safety remains the most common vulnerability category (heap out-of-bounds read/write)

Top root causes since 2016: #1: heap out-of-bounds | #2: use after free | #3: type confusion | #4: uninitialized use

Lessons Learned

- **Low-level resources are shared mutable states.**

- **Shared mutable states break modularity.**
  - Too many things are intertwined in shared mutable states.
  - Often far-away and even inaccessible code (e.g., library) is involved.

- **Lack of modularity hinders construction of large software.**
  - Human simply cannot think of everything all at once.

- **Reasoning principle for shared mutable states is necessary.**
  - We want to achieve modularity.
  - Human probably cannot achieve this without automated tools.

Reasoning Shared Mutable States, Previously

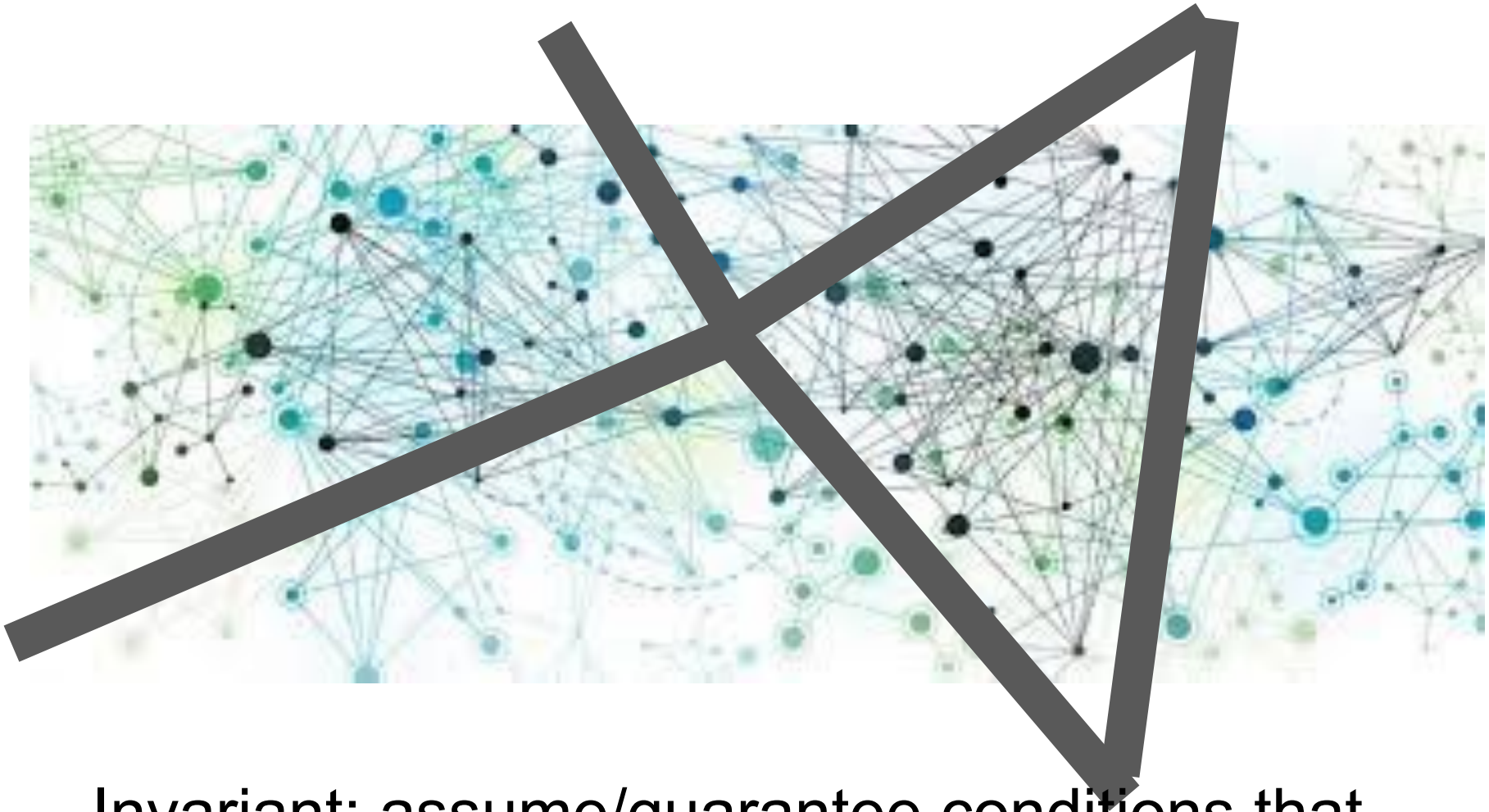
A Wisdom on Shared Mutable States

- **Linus Torvalds (2006, https://lwn.net/Articles/193245/)**
  - I will, in fact, claim that the difference between a bad programmer and a good one is whether [s/]he considers [her/]his code or [her/]his data structures more important. Bad programmers worry about the code. **Good programmers worry about data structures and their relationships.**

State

“Invariant”

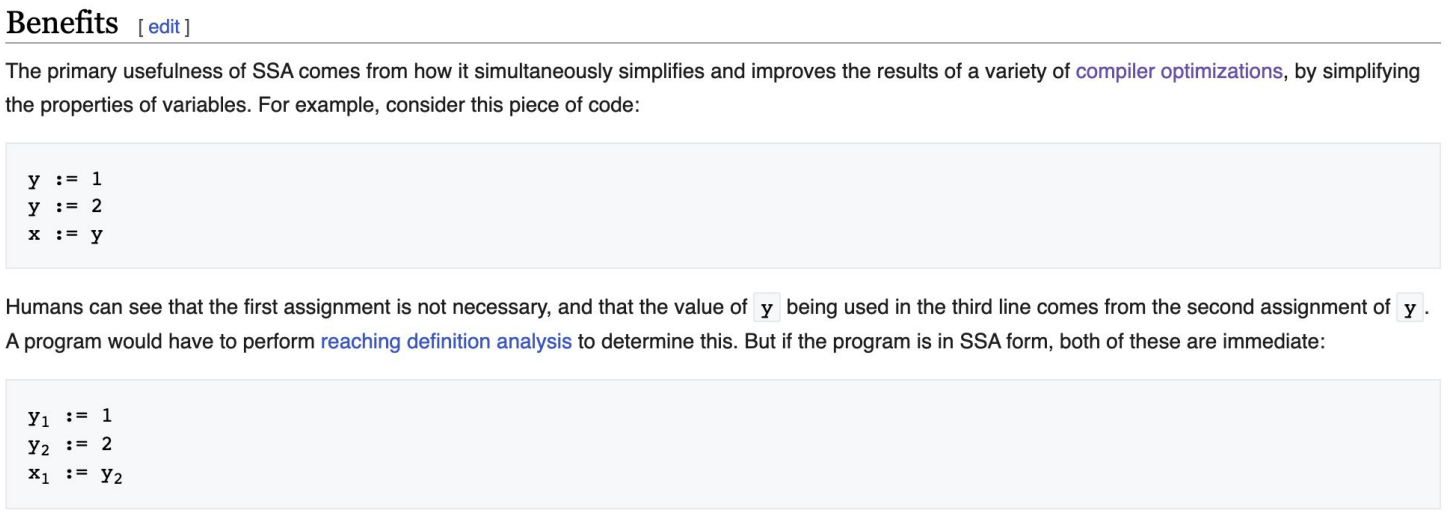
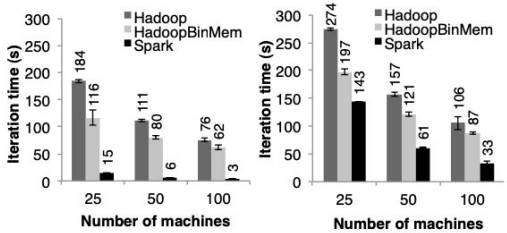
Invariant Enables Modular Reasoning



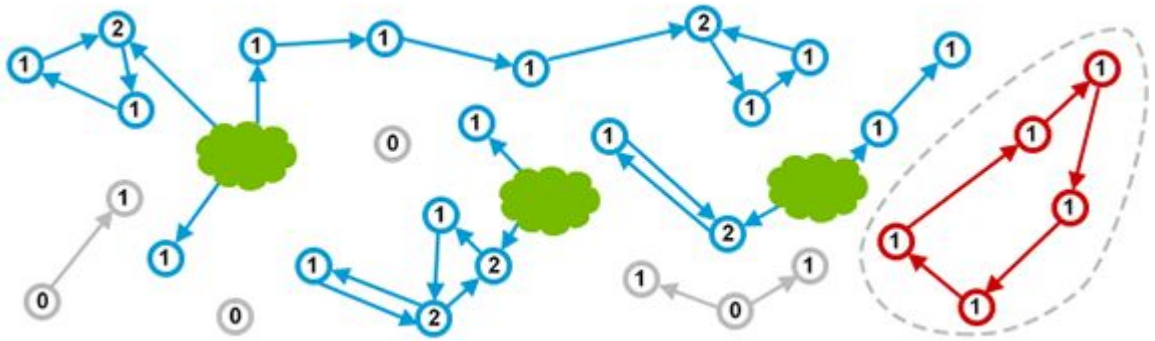
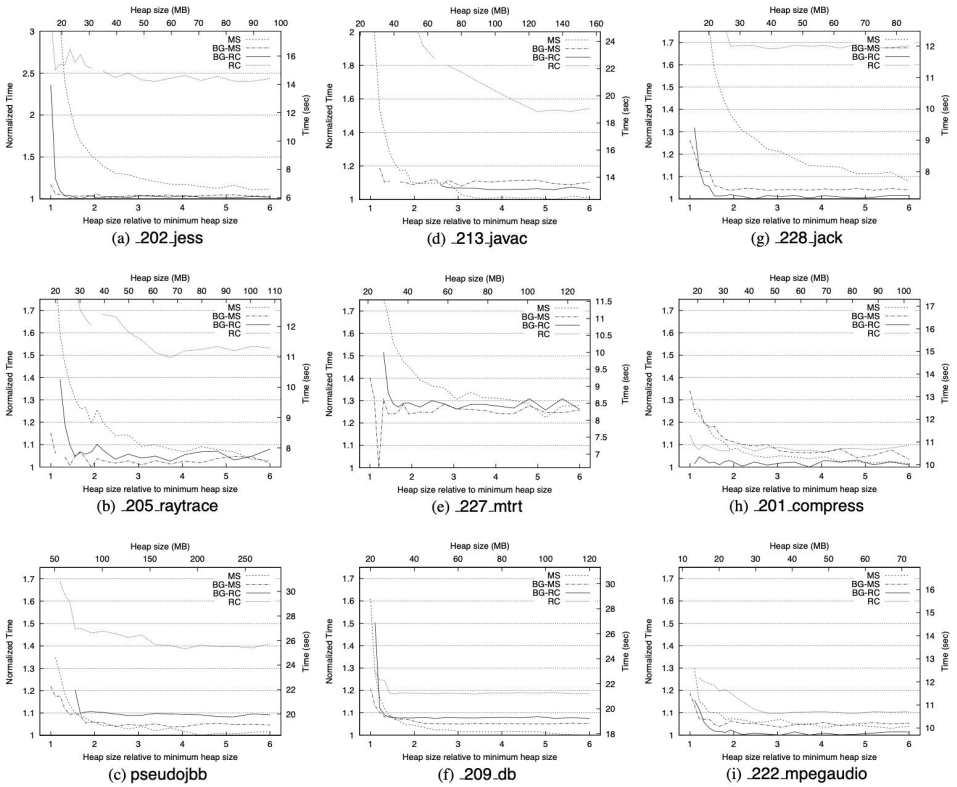
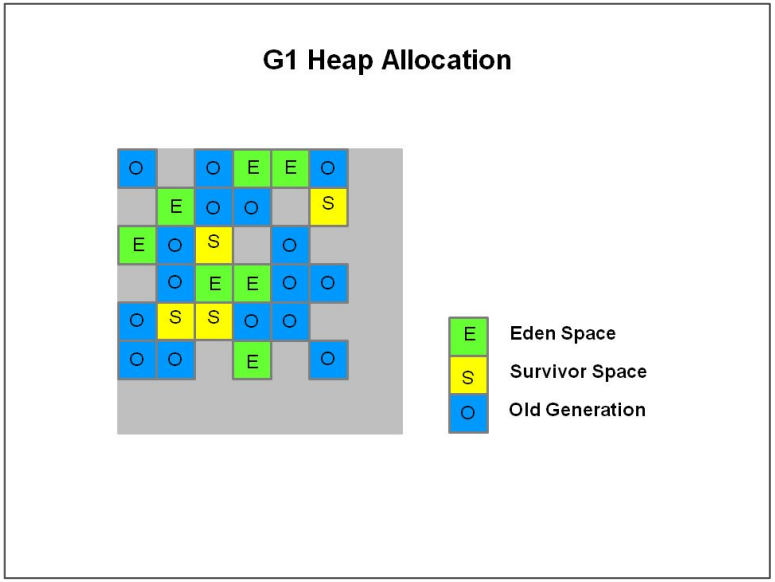
Invariant: assume/guarantee conditions that “divides” complex systems

Invariant Examples

- **How a state's components relates among each other? (spatial)**
- **How a state evolves over time? (temporal)**

- **E.g., Immutability**
  - This object should be 42 always.
  - `const auto x = 42;`
- **E.g., No sharing**
  - This object is referenced only by this variable.
  - ```
    auto x = 42; // 42
    x = 666;     // 666
    x = 37;      // 37
    ```
- **More complex invariants for larger objects**
  - `x` is the head of a sorted linked list whose nodes are …
    ```

Invariant-based Modular Reasoning

- **No shared mutable states**
  - Immutability

```
const auto x = 42;
…
x; // must be 42
```

  - No sharing

```
auto x = 42;
…  // x is not shared and I didn't change x
x; // must be 42
```

- **Tamed shared mutable states**
  - Concurrent sorted linked list: I can safely traverse to the next node *because* each node's next pointer is valid.

Immutability (Invariant)

- **Shared *Immutable* States**
  - Unconstrained sharing
  - Often accompanied with garbage collection
    - A tracking of (unconstrained) sharing is necessary
- **Best fit for functional languages**
  - Immutability by default
  - E.g., OCaml, Haskell
- **Good fit for parallel & distributed systems**
  - Immutability for performance (less cache invalidation & synchronization)
  - E.g., Immutable & functional data structures

Immutability Example: SSA (POPL 1988)

- **“Static Single Assignment”**
  - Compiler optimization technique
  - Invariant: “a variable is assigned only once in a program.”
- **Benefits**
  - Modularity: (relatively) immutable after assignment
  - 

Benefits [edit]

The primary usefulness of SSA comes from how it simultaneously simplifies and improves the results of a variety of compiler optimizations, by simplifying the properties of variables. For example, consider this piece of code:

```
y := 1
y := 2
x := y
```

Humans can see that the first assignment is not necessary, and that the value of `y` being used in the third line comes from the second assignment of `y`. A program would have to perform reaching definition analysis to determine this. But if the program is in SSA form, both of these are immediate:

```
y1 := 1
y2 := 2
x1 := y2
```

Immutability Example: RDD (NSDI 2012)

- **"Resilient Distributed Dataset"**
  - Immutable by definition
- **Benefits**
  - simple programming model

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
errors.persist()
```

  - that achieves high performance



Figure 8: Running times for iterations after the first in Hadoop, HadoopBinMem, and Spark. The jobs all processed 100 GB.

Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing

Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, Ion Stoica
University of California, Berkeley

Abstract

We present Resilient Distributed Datasets (RDDs), a distributed memory abstraction that lets programmers perform in-memory computations on large clusters in a fault-tolerant manner. RDDs are motivated by two types of applications that current computing frameworks handle inefficiently: iterative algorithms and interactive data mining tools. In both cases, keeping data in memory can improve performance by an order of magnitude. To achieve fault tolerance efficiently, RDDs provide a restricted form of shared memory, based on coarse-grained transformations rather than fine-grained updates to shared state. However, we show that RDDs are expressive enough to capture a wide class of computations, including recent specialized programming models for iterative jobs, such as Pregel, and new applications that these models do not capture. We have implemented RDDs in a system called Spark, which we evaluate through a variety of user applications and benchmarks.

1 Introduction

Cluster computing frameworks like MapReduce [10] and Dryad [19] have been widely adopted for large-scale data analytics. These systems let users write parallel computations using a set of high-level operators, without having to worry about work distribution and fault tolerance.

Although current frameworks provide numerous abstractions for accessing a cluster's computational resources, they lack abstractions for leveraging distributed memory. This makes them inefficient for an important class of emerging applications: those that *reuse* intermediate results across multiple computations. Data reuse is common in many *iterative* machine learning and graph algorithms, including PageRank, K-means clustering, and logistic regression. Another compelling use case is *interactive* data mining, where a user runs multiple ad-hoc queries on the same subset of the data. Unfortunately, in most current frameworks, the only way to reuse data between computations (*e.g.,* between two MapReduce jobs) is to write it to an external stable storage system, *e.g.,* a distributed file system. This incurs substantial overheads due to data replication, disk I/O, and serialization, which can dominate application execution times.

Recognizing this problem, researchers have developed specialized frameworks for some applications that require data reuse. For example, Pregel [22] is a system for iterative graph computations that keeps intermediate data in memory, while HaLoop [7] offers an iterative MapReduce interface. However, these frameworks only support specific computation patterns (*e.g.,* looping a series of MapReduce steps), and perform data sharing implicitly for these patterns. They do not provide abstractions for more general reuse, *e.g.,* to let a user load several datasets into memory and run ad-hoc queries across them.

In this paper, we propose a new abstraction called *resilient distributed datasets (RDDs)* that enables efficient data reuse in a broad range of applications. RDDs are fault-tolerant, parallel data structures that let users explicitly persist intermediate results in memory, control their partitioning to optimize data placement, and manipulate them using a rich set of operators.

The main challenge in designing RDDs is defining a programming interface that can provide fault tolerance *efficiently*. Existing abstractions for in-memory storage on clusters, such as distributed shared memory [24], key-value stores [25], databases, and Piccolo [27], offer an interface based on fine-grained updates to mutable state (*e.g.,* cells in a table). With this interface, the only ways to provide fault tolerance are to replicate the data across machines or to log updates across machines. Both approaches are expensive for data-intensive workloads, as they require copying large amounts of data over the cluster network, whose bandwidth is far lower than that of RAM, and they incur substantial storage overhead.

In contrast to these systems, RDDs provide an interface based on *coarse-grained* transformations (*e.g.,* map, filter and join) that apply the same operation to many data items. This allows them to efficiently provide fault tolerance by logging the transformations used to build a dataset (its *lineage*) rather than the actual data.[1] If a partition of an RDD is lost, the RDD has enough information about how it was derived from other RDDs to recompute

[1] Checkpointing the data in some RDDs may be useful when a lineage chain grows large, however, and we discuss how to do it in §5.4.

But Not Everything is Immutable

- **Iterator invalidation (again): we want to mutate the vector.**
  - ```
    #include <bits/stdc++.h>
    using namespace std;
    int main() {
        vector<int> v = { 1, 5, 10, 15, 20 };
        for (auto it = v.begin(); it != v.end(); it++)
            if ((*it) == 5)
                v.push_back(-1);
        for (auto it = v.begin(); it != v.end(); it++)
            cout << (*it) << " ";
        return 0;
    }
    ```
- **In general, we want mutate states to make visible effects.**
    ```

No Sharing (Invariant)

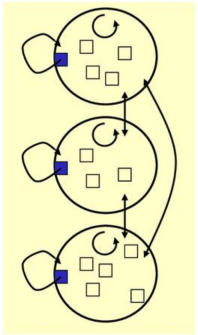
- **Example**
  - ```
    auto x = 42;
    … // x is not shared and I didn't change x
    x; // must be 42
    ```
- **Why important?**
  - Typical cache hit ratio > 99% → most data are exclusively accessed
  - Exclusive access → modular understanding & more optimizations
    ```

No Sharing Example: Alias Analysis

- **Two pointers are aliased?**
  - Exactly? Overlapped?
  - Always? Possibly?
- **Basis for optimizations**
  - Constant propagation
  - Value numbering
  - Redundancy elimination
  - Code motion
- **Basis for understanding**
  - No interference from the other functions or threads (possibly unknown)!

- **… but sharing happens in the real-world programs.**

Principled Sharing Example: Reference Counting

- **Each object has a counter for incoming references.**
  - It's incremented/decremented when a reference is created/destroyed.
  - Once it reaches zero, the object can be reclaimed.
- **Runtime overhead due to…**
  - Counting
  - "Detached group" (yet unreclaimed)



https://plumbr.io/handbook/what-is-garbage-collection/automated-memory-management/reference-counting

But Not All Programs Can Admit RC

- **Reference counting is slow.**



Figure 7: Total Time as a Function of Heap Size

Blackburn and McKinley. Ulterior reference counting: fast garbage collection without a long wait. OOPSLA 2003.

Principled Sharing Example: Garbage Collection

- **Automatically cleaning up no-longer-used memory locations**
  - Tracing (which locations are reachable?)
  - Sweeping (let's reclaim unreachable objects.)
  - Compacting (let's move objects to reserve contiguous memory region.)
- **Safety & Impressive throughput**
  - Sometime outperforming C/C++ (5-instruction allocation!)
  - With strong safety guarantee
- **Limitations**
  - Not precise control of reclamation
  - Higher memory usage
  - Unpredictable-ish pause



https://www.oracle.com/technetwork/tutorials/tutorials-1876574.html

Wisdom: if GC works for you, just use it.

Patrick Walton @pcwalton · 2021년 11월 14일

What people who say they want a borrow check for C/C++/Zig/etc really want is a system that will automatically achieve memory safety without giving the programmer strict rules to follow. That technology exists and is very mature: **garbage collection**.

Patrick Walton @pcwalton · 2018년 3월 27일

Boring truth about **garbage collection**: The design of the GC of the HotSpot JVM is mostly optimal.

Patrick Walton
@pcwalton

My strong feeling is that the "memory safe systems programming language that's easier to use than Rust" that everyone wants is going to have to have something that looks a lot like a garbage collector. There's a lot of room to design a fast GC for "systems-like" patterns.

트윗 번역하기

오전 7:39 · 2022년 7월 28일

OOPSLA 2005

Quantifying the Performance of Garbage Collection vs. Explicit Memory Management

Matthew Hertz *
Computer Science Department
Canisius College
Buffalo, NY 14208
matthew.hertz@canisius.edu

Emery D. Berger
Dept. of Computer Science
University of Massachusetts Amherst
Amherst, MA 01003
emery@cs.umass.edu

ABSTRACT

Garbage collection yields numerous software engineering benefits, but its quantitative impact on performance remains elusive. One can compare the cost of *conservative* garbage collection to explicit memory management in C/C++ programs by linking in an appropriate collector. This kind of direct comparison is not possible for languages designed for garbage collection (e.g., Java), because programs in these languages naturally do not contain calls to `free`. Thus, the actual gap between the time and space performance of explicit memory management and *precise*, copying garbage collection remains unknown.

We introduce a novel experimental methodology that lets us quantify the performance of precise garbage collection versus explicit memory management. Our system allows us to treat unaltered Java programs as if they used explicit memory management by relying on oracles to insert calls to `free`. These oracles are generated from profile information gathered in earlier application runs. By executing inside an architecturally-detailed simulator, this "oracular" memory manager eliminates the effects of consulting an oracle while measuring the costs of calling `malloc` and `free`. We evaluate two different oracles: a liveness-based oracle that aggressively frees objects immediately after their last use, and a reachability-based oracle that conservatively frees objects just after they are last reachable. These oracles span the range of possible placement of explicit deallocation calls.

We compare explicit memory management to both copying and non-copying garbage collectors across a range of benchmarks using the oracular memory manager, and present real (non-simulated) runs that lend further validity to our results. These results quantify the time-space tradeoff of garbage collection: with five times as much memory, an Appel-style generational collector with a non-copying mature space matches the performance of reachability-based explicit memory management. With only three times as much memory, the collector runs on average 17% slower than explicit memory management. However, with only twice as much memory, garbage collection degrades performance by nearly 70%. When physical memory is scarce, paging causes garbage collection to run an order of magnitude slower than explicit memory management.

Categories and Subject Descriptors

D.3.3 [**Programming Languages**]: Dynamic storage management; D.3.4 [**Processors**]: Memory management (garbage collection)

General Terms

Experimentation, Measurement, Performance

Keywords

oracular memory management, garbage collection, explicit memory management, performance analysis, time-space tradeoff, throughput, paging

1. Introduction

Garbage collection, or automatic memory management, provides significant software engineering benefits over explicit memory management. For example, garbage collection frees programmers from the burden of memory management, eliminates most memory leaks, and improves modularity, while preventing accidental memory overwrites ("dangling pointers") [50, 59]. Because of these advantages, garbage collection has been incorporated as a feature of a number of mainstream programming languages.

Garbage collection can improve programmer productivity [48], but its impact on performance is difficult to quantify. Previous researchers have measured the runtime performance and space impact of *conservative*, non-copying garbage collection in C and C++ programs [19, 62]. For these programs, comparing the performance of explicit memory management to conservative garbage collection is a matter of linking in a library like the Boehm-Demers-Weiser collector [14]. Unfortunately, measuring the performance trade-off in languages designed for garbage collection is not so straightforward. Because programs written in these languages do not explicitly deallocate objects, one cannot simply replace garbage collection with an explicit memory manager. Extrapolating the results of studies with conservative collectors is impossible because precise, relocating garbage collectors (suitable only for garbage-collected languages) consistently outperform conservative, non-relocating garbage collectors [10, 12].

*Work performed at the University of Massachusetts Amherst.

But Not All Programs Can Admit GC

- **Time-space tradeoff (Hertz and Berger, OOPSLA 2005)**
  - [W]ith five times as much memory, an Appel-style generational collector with a non-copying mature space matches the performance of reachability-based explicit memory management.
  - With only three times as much memory, the collector runs on average 17% slower than explicit memory management.
  - However, with only twice as much memory, garbage collection degrades performance by nearly 70%.
  - When physical memory is scarce, paging causes garbage collection to run an order of magnitude slower than explicit memory management.
- **GC (relatively) lacks precise control over non-memory resources**
  - “Systems programming”

Lessons Learned

- **We want invariant that tames shared mutable states.**
  - To achieve modularity automatically

- **Both of immutability & no/principled sharing are great invariants.**
  - Huge impact on real-world products & a lot of research papers
  - Though each misses some crucial use cases.

- **How to more precisely reason about shared mutable states?**
  - Automatically
  - More time/space-efficiently than RC and GC

Assignment (Day 2)

- **Read the Rust Book Chapters 1-10 (Strongly Recommended)**
  - https://doc.rust-lang.org/book/
  - If you cannot understand something, feel free to skip it.

- **Solve KAIST CS220 Homework 7 (Strongly Recommended)**
  - https://github.com/kaist-cp/cs220/tree/main/src/assignments
  - https://gg.kaist.ac.kr/course/14/
  - Small but deep

- **Solve KAIST CS431 Homework 1 (Optional)**
  - https://github.com/kaist-cp/cs431/issues/606
  - https://gg.kaist.ac.kr/course/14/

Focus on memory

Reasoning Shared Mutable States, Efficiently & Precisely

Type checking
(automatically w/o runtime overhead)

Invariant

Review: Memory

- **Memory: map from location to value**
  - a primary example of shared mutable states
  - Each location is a shared mutable state
- **Stack and heap regions**
  - Stack: automatically allocated/freed when a function is started/finished
  - Heap: manually allocated/freed
- **Pointer: general accessor to locations**
  - Load, store, free
  - A location object may be shared among multiple pointers

- **Goal: reasoning about the safety of each pointer access**

Review: Memory Safety

- `auto a = (int *) malloc(sizeof(int));`
  `*a = 42;`
  `auto b = a;`
  `std::thread t([=](){ *b = 666; });`

- **Data race**
  - `auto x = *a; // 42? 666? unconstrained non-determinism`
- **Use-after-free**
  - `free(a);`
- **Double free**
  - `auto c = a;`
    `std::thread t([=](){ free(c); });`
    `free(a);`

Focus on UAF

Type-Checking Memory Safety in C/C++ (Fail)

- **Attempt 1**
  - ```
    void foo(int *p) {
        *p; // Potentially unsafe: maybe NULL
    }
    ```
  - Lesson: we need to *enrich* pointer types with an invariant.
- **Attempt 2**
  - ```
    void foo(int &p) {
        p; // Potentially unsafe: (int *) ~= (int &) in C/C++
    }
    ```
  - Lesson: we need to escape C.
- **Attempt 3**
  - ```
    void foo(unique_ptr<int> p) { // unique_ptr: implying uniqueness of p
        *p; // Potentially unsafe: “auto q = std::unique_ptr<int>{p.get()};”
    }
    ```
  - Lesson: we need to escape C++ and embrace a more strict type checker.
    ```

Type-Checking Memory Safety in New Language

- **An early attempt: Cyclone (PLDI 2002)**
  - Not widely adopted, though
  - [Huge influence to Rust]

Regions

- a.k.a. zones, arenas, …
- Every object is in exactly one region
- All objects in a region are deallocated simultaneously (no `free` on an object)
- Allocation via a region *handle*



An old idea with some support in languages (e.g., RC) and implementations (e.g., ML Kit)

Region-Based Memory Management in Cyclone *

Dan Grossman Greg Morrisett Trevor Jim†
Michael Hicks Yanling Wang James Cheney

Computer Science Department
Cornell University
Ithaca, NY 14853
{danieljg,jgm,mhicks,wangyl,jcheney}@cs.cornell.edu

†AT&T Labs Research
180 Park Avenue
Florham Park, NJ 07932
trevor@research.att.com

ABSTRACT

Cyclone is a type-safe programming language derived from C. The primary design goal of Cyclone is to let programmers control data representation and memory management without sacrificing type-safety. In this paper, we focus on the region-based memory management of Cyclone and its static typing discipline. The design incorporates several advancements, including support for region subtyping and a coherent integration with stack allocation and a garbage collector. To support separate compilation, Cyclone requires programmers to write some explicit region annotations, but a combination of default annotations, local type inference, and a novel treatment of region effects reduces this burden. As a result, we integrate C idioms in a region-based framework. In our experience, porting legacy C to Cyclone has required altering about 8% of the code; of the changes, only 6% (of the 8%) were region annotations.

Categories and Subject Descriptors

D.3.3 [**Programming Languages**]: Language Constructs and Features—*dynamic storage management*

General Terms

Languages

1. INTRODUCTION

Many software systems, including operating systems, device drivers, file servers, and databases require fine-grained control over data representation (e.g., field layout) and resource management (e.g., memory management). The *de facto* language for coding such systems is C. However, in providing low-level control, C admits a wide class of dangerous — and extremely common — safety violations, such as incorrect type casts, buffer overruns, dangling-pointer dereferences, and space leaks. As a result, building large systems in C, especially ones including third-party extensions, is perilous. Higher-level, type-safe languages avoid these drawbacks, but in so doing, they often fail to give programmers the control needed in low-level systems. Moreover, porting or extending legacy code is often prohibitively expensive. Therefore, a safe language at the C level of abstraction, with an easy porting path, would be an attractive option.

Toward this end, we have developed *Cyclone* [6, 19], a language designed to be very close to C, but also safe. We have written or ported over 110,000 lines of Cyclone code, including the Cyclone compiler, an extensive library, lexer and parser generators, compression utilities, device drivers, a multimedia distribution overlay network, a web server, and many smaller benchmarks. In the process, we identified many common C idioms that are usually safe, but which the C type system is too weak to verify. We then augmented the language with modern features and types so that programmers can still use the idioms, but have safety guarantees.

For example, to reduce the need for type casts, Cyclone has features like parametric polymorphism, subtyping, and tagged unions. To prevent bounds violations without making hidden data-representation changes, Cyclone has a variety of pointer types with different compile-time invariants and associated run-time checks. Other projects aimed at making legacy C code safe have addressed these issues with somewhat different approaches, as discussed in Section 7.

In this paper, we focus on the most novel aspect of Cyclone: its system for preventing dangling-pointer dereferences and space leaks. The design addresses several seemingly conflicting goals. Specifically, the system is:

- *Sound:* Programs never dereference dangling pointers.
- *Static:* Dereferencing a dangling pointer is a compile-time error. No run-time checks are needed to determine if memory has been deallocated.
- *Convenient:* We minimize the need for explicit programmer annotations while supporting many C idioms. In particular, many uses of the addresses of local variables require no modification.

*This research was supported in part by Sloan grant BR-3734; NSF grant 9875536; AFOSR grants F49620-00-1-0198, F49620-01-1-0298, F49620-00-1-0209, and F49620-01-1-0312; ONR grant N00014-01-1-0968; and NSF Graduate Fellowships. Any opinions, findings, and conclusions or recommendations expressed in this publication are those of the authors and do not reflect the views of these agencies.

PLDI'02, June 17-19, 2002, Berlin, Germany.

Type-Checking Memory Safety in Rust

- **Necessary for types to represent the “live pointer” invariant**
- **Multiple such invariants**
  - w/ different conditions on mutability & sharing
  - ```
    fn foo(p: Box<i32>) { // assume: p "owns" an object
        *p;               // guarantee: always safe
    }
    ```
  - ```
    fn foo(p: &i32) {     // assume: p "borrows" an object
        *p;               // guarantee: always safe
    }
    ```
  - ```
    fn foo(p: &mut i32) { // assume: p "mutably borrows" an object
        *p;               // guarantee: always safe
    }
    ```
    ```

Owned Type (Mutable & No Sharing)

- **Example: Box**
  - ```
    fn foo(p: Box<i32>) { // assume: p "owns" an object
        *p;               // guarantee: always safe
    }
    ```
- **Transferable & automatically reclaimed (even for heap)**
  - ```
    fn qux() {
        let p = Box::new(42i32);
        let q = p; // object's ownership transferred to "q"; "p" invalidated
    } // "q" reclaimed
    ```
  - ```
    fn bar() {
        let p = Box::new(42i32);
        foo(p); // object's ownership transferred to "foo"; "p" is
    invalidated
    } // quiz: "p" is reclaimed where?
    ```
    ```

Borrowed Type (Immutable & Full Sharing, 1/3)

- **Example: reference**
  - ```
    fn main() {
        let s1 = String::from("hello");  // ownership
        let len = calculate_length(&s1); // borrow
        println!("The length of '{}' is {}.", s1, len);
    }
    fn calculate_length(s: &String) -> usize {
        s.len() // Why safe?
    }
    ```
  - Safe because “s” is dereferenced while “s1” is valid (not yet reclaimed).
  - Vocabulary: reference “s” immutably borrows from object “s1”.
- **No constraints on the degree of sharing**
    ```

Borrowed Type (Immutable & Full Sharing, 2/3)

- **Type checking: ensuring pointer validity**
  - ```
    fn main() {
        let p = Box::new(42i32);
        let x = &p;
        drop(p);
        *x; // ERROR: use-after-free
    }
    ```
  - ```
    fn main() {
        let reference_to_nothing = dangle();
    }
    fn dangle() -> &String {
        let s = String::from("hello");
        &s
    } // ERROR: s is dropped, return is dangling!
    ```

Borrowed Type (Immutable & Full Sharing, 3/3)

- **Algorithm: ensuring pointer validity with lifetime analysis**
  - If a borrower lives longer than its owner, REJECT!

```
fn main() {
    let r;                // ---------+-- 'a
                          //          |
    {                     //          |
        let x = 5;        // -+-- 'b  |
        r = &x;           //  |       |
    }                     // -+       |
                          //          |
    println!("r: {}", r); //          |
}                         // ---------+
```


```
fn main() {
    let x = 5;            // ----------+-- 'b
                          //           |
    let r = &x;           // --+-- 'a  |
                          //   |       |
    println!("r: {}", r); //   |       |
                          // --+       |
}                         // ----------+
```

https://doc.rust-lang.org/book/ch10-03-lifetime-syntax.html

Mutably Borrowed Type (Mutable & Sharing, 1/3)

- **Motivation: we want to modify data w/ borrows.**
  - ```
    fn main() {
        let s = String::from("hello");
        let s = change(s); // OK, but cumbersome…
    }
    fn change(mut some_string: String) -> String {
        some_string.push_str(", world");
        Some_string
    }
    ```
  - ```
    fn main() {
        let s = String::from("hello");
        change(&s);
    }
    fn change(some_string: &String) {
        some_string.push_str(", world"); // ERROR! Breaking immutability
    }
    ```

Mutably Borrowed Type (Mutable & Sharing, 2/3)

- **Example: mutable reference**
  - ```
    fn main() {
        let mut s = String::from("hello");
        change(&mut s);
    }
    fn change(some_string: &mut String) {
        some_string.push_str(", world"); // OK
    }
    ```
- **Benefits**
  - More readable than ownership transfer version
  - More efficient: no data transfer; only pointer transfer
    ```

Mutably Borrowed Type (Mutable & Sharing, 3/3)

- **Type checking: preventing concurrent mutation**
  - Recall: shared mutable state is the evil.
  - Data race, iterator invalidation, …
- **Algorithm: enforcing A⊕M Principle**
  - “Alias (borrow) XOR mutation”
  - If an object is borrowed, it cannot be mutated.

```
fn calculate_length(s: &String) -> usize {
    s.push_str("covfefe"); // Compile error: cannot mutate &String
}
```

  - If an object is not borrowed, it can be mutated.

```
fn main() {
    let mut s1 = String::from("hello");
    s1.push_str("covfefe"); // OK
}
```

Summary of Pointer Types in Rust (1/2)

- **Owned (mutable & no sharing)**
  - Exclusive permission to access an object
  - Transferable, immutably & mutably borrowable

- **Immutably borrowed (immutable & full sharing)**
  - Immutably borrowable (“reborrow”)

- **Mutably borrowed (mutable & constrained sharing)**
  - Immutably & mutably borrowable (“reborrow”)

Summary of Pointer Types in Rust (2/2)

- **Automatically ensuring pointer safety (valid & A⊕M)**
  - Lifetime analysis: ensuring pointer validity of borrows
  - A⊕M analysis: ensuring no concurrent mutable borrows

- **Preventing memory safety errors**
  - Data race: write requires [ownership or mutable borrow] (exclusive)
  - Double free: free requires ownership (exclusive)

- **Representing “flow-sensitive” invariant**
  - The same pointer have different types in different program points
  - Ownership → mutable borrow → borrow → ownership → …

Remaining Question: How Much Applicable?

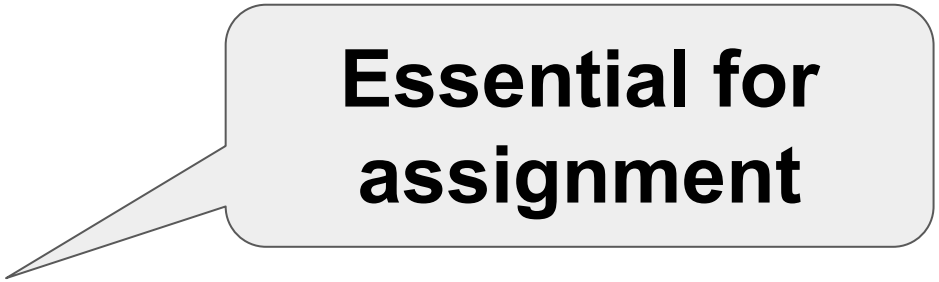
- **Applicable to more programming features?**
  - Slices and structs
  - Smart pointers
  - first-class functions
  - Parallel programming
  - …

- **Spoiler: We'll need hybrid (static + runtime) verification…**
  - "Unsafe Rust"

Whirlwind Tour of Rust Type System

Summary

- **Motivation: learning how to “communicate” with the Rust type checker**
- **References**
  - https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html
  - https://doc.rust-lang.org/book/ch10-03-lifetime-syntax.html
- **Key points (focus on examples)**
  - “Ownership Rules”
  - “Variables and Data Interacting with Move / Clone”
  - “Stack-Only Data: Copy”
  - “Return Values and Scope”
  - “Dangling References”
  - “The Rules of References”
  - **“The Slice Type” (slice)**
  - **“Validating References with Lifetimes” (struct)**



Assignment (Day 3)

- **Solve KAIST CS220 Homework 11 (Strongly Recommended)**
  - https://github.com/kaist-cp/cs220/tree/main/src/assignments
  - https://gg.kaist.ac.kr/course/14/
  - Large but shallow

- **Solve KAIST CS431 Homework 1 (Strongly Recommended)**
  - https://github.com/kaist-cp/cs431/issues/606
  - https://gg.kaist.ac.kr/course/14/

Reasoning Shared Mutable States In Functions

Review: First-Class Function

- **Definition: function as a value**
  - ```
    let expensive_closure = |num: u32| -> u32 {
        println!("calculating slowly...");
        thread::sleep(Duration::from_secs(2));
        Num
    };
    ```
- **Capturing variables**
  - ```
    fn main() {
      let list = vec![1, 2, 3];
      println!("Before defining closure: {:?}", list);
      thread::spawn(move || println!("From thread: {:?}",
    list)).join().unwrap();
    }
    ```

Question: What If a Function Captures Pointer?

- **First-class function may capture pointers…**
  - ```
    let list = vec![1, 2, 3];
    let only_borrows = || println!("From closure: {:?}", list);
    println!("Before calling closure: {:?}", list);
    only_borrows();
    println!("After calling closure: {:?}", list);
    ```
- **The same rule as pointer-capturing structs**
  - ```
    struct User<'a> {
        active: bool,
        username: &'a str,
        email: &'a str,
        sign_in_count: u64,
    }
    ```
  - Ownership transfer, immutable borrow, or mutable borrow
- **Reference (focus on examples)**
  - https://doc.rust-lang.org/book/ch13-01-closures.html

Reasoning Shared Mutable States With Smart Pointers

What is Smart Pointer?

- **Data structures that act like a pointer**
  - Box: heap allocation (unique_ptr in C++)
  - Rc: reference counter (sequential)
  - Arc: reference counter (multi-threaded, A for “atomic”)
  - RefCell: runtime borrows (e.g., try_borrow(), try_borrow_mut())
- **Beyond the scope of Rust’s static verification (valid & A⊕M)**
  - Box: dereferences an internal raw pointer
  - Arc: mutates the reference counter concurrently by multiple threads
  - Rc: mutates the reference counter even in the presence of other refs
  - RefCell: effectively promotes “&” to “&mut”

Smart Pointer's Runtime Verification

- **Runtime verification for pointer safety (valid & A⊕M)**
  - Box: ownership → exclusive permission to a live pointer in heap
  - Rc/Arc: counter > 0 → safe to dereference
  - RefCell: runtime verification of A⊕M Principle

- **Rust Book (focus on examples)**
  - https://doc.rust-lang.org/book/ch15-00-smart-pointers.html

Supporting Runtime Verification w/ Unsafe Rust

- **Question: how to implement smart pointers in Rust?**
  - At the first place, they don't pass Rust's static verification!

- **Answer: "unsafe" escape hatch**
  - Raw pointer arithmetic & dereference
  - Unchecked pointer access (e.g., unchecked array indexing)
  - `unsafe { very_unsafe_op(); }`
  - Example (RefCell): https://doc.rust-lang.org/src/core/cell.rs.html#994-996

- **… Then what's the difference from C/C++?**

Improving Modularity w/ Interior Mutability

- **Interior mutability: enveloping potential unsafety within safe API**
  - **Impl** of RefCell::try_borrow(): converting “&” to “&mut” (unsafe Rust)
  - **API**: “as if” the client doesn’t mutate RefCell (safe Rust)
  - **Obligation**: library designer’s manual reasoning (as a comment)
  - **Reference**: https://doc.rust-lang.org/book/ch15-05-interior-mutability.html

- **Better modularity**
  - Clients don’t need to care potentially unsafe impl (“far-away” mutations)
  - C/C++ lacks such modular reasoning support for potential unsafety

Hybriding Static and Runtime Verification

- **Runtime verification**
  - (Reference counter == 1) → safe to get “&mut”
- **Static verification**
  - From “&mut”, we can get multiple “&”
- **Smooth composition of static + runtime verification**
  - ```
    fn foo(p: &mut i32) { // static verification
        let q = &*p;
        let r = &*p;
    }
    fn main() {
        let mut x = Rc::new(42i32);
        let p = Rc::get_mut(&mut x).unwrap();  { // runtime verification
        foo(p);
    }
    ```
  - Reason: “&mut” serves as the invariant enabling modular reasoning
    ```

Reasoning Shared Mutable States In Parallel Programming

Concurrency API (1/3)

- **Thread spawn**
  - Reference: https://doc.rust-lang.org/std/thread/fn.spawn.html

Function std::thread::spawn

```
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T + Send + 'static,
    T: Send + 'static,
```

  - ‘static: shouldn’t pass immortal references
  - Send: multi-thread safe

Concurrency API (2/3)

- **Send**
  - “ownership of values of the type implementing Send can be transferred between threads.” [link]
  - Examples: Box, Mutex, Arc, RefCell, …
  - Non-examples: Rc [link]
- **Sync**
  - “any type T is Sync if &T (an immutable reference to T) is Send, meaning the reference can be sent safely to another thread.” [link]
  - Examples: Box, Mutex, Arc
  - Non-exmaples: Rc, RefCell [link]

Concurrency API (3/3)

- **Channel**
  - Reference [link]

Function std::sync::mpsc::channel 1.0.0 ·

```
pub fn channel<T>() -> (Sender<T>, Receiver<T>)
```

  - Wait, T must be Send, right? (Not exactly, but close…)

```
impl<T: Send> Send for Sender<T>

impl<T> !Sync for Sender<T>
```

Parallelism API

- **Rayon: Data-parallelism library**
  - Benefit: no concurrency API used explicitly (except for Send/Sync)
- **Examples**
  - ```
    use rayon::prelude::*;
    fn sum_of_squares(input: &[i32]) -> i32 {
        input.par_iter()
             .map(|i| i * i)
             .sum()
    }
    ```
  - https://docs.rs/rayon/latest/rayon/iter/trait.ParallelIterator.html
  - https://docs.rs/rayon/latest/rayon/iter/trait.IndexedParallelIterator.html
  - https://docs.rs/rayon/latest/rayon/slice/trait.ParallelSliceMut.html#method.par_sort
    ```

Epilogue

Recall: Organization

- **Why it's hard to ensure safety of low-level control? (day 1)**
  - "Shared mutable states"
  - Existing approaches to (automatically) ensure safety
- **What's the key idea of Rust? (day 1, 2)**
  - Static verification
  - "Static": compile-time (automatic, no runtime overhead)
- **How to apply the key idea to programming features? (day 3)**
  - Smart pointers & hybrid (static + runtime) verification of safety
  - First-class functions
  - Parallel programming

Rust in FuriosaAI

- **NPU chip startup @ Seoul**
  - https://furiosa.ai
  - First chip Warboy available in Q1 2023

- **Rust is widely used in the software stack**
  - From 2017
  - Compiler, serving system, program visualizer, profiler, …
  - Huge productivity gain thanks to Rust's type checker

- **Client API in Rust**
  - https://github.com/orgs/furiosa-ai/repositories

Rust in KAIST

- **Concurrency and Parallelism Laboratory**
  - https://cp.kaist.ac.kr
  - CS220: Programming Principles [link]
  - CS420: Compiler Design [link]
  - CS431: Concurrent Programming [link]
  - Hardware description language (ASPLOS 2023)
  - Garbage collectors (PLDI 2020, submitted)
  - Persistent data structures (submitted)
  - NPU serving systems (submitted)
- **Computer Architecture and Systems Laboratory**
  - http://casys.kaist.ac.kr/
  - CS492: Virtualization
  - Isolation of system software components using Rust (submitted)

Rust in SNU?

- **I wish you'll also enjoy huge productivity gain of Rust.**
  - The most loved language (Stack Overflow survey)
  - The only programming language I can speak

- **I'm widely open to collaboration.**
  - If you plan to use Rust in your project, I wish to hear from you.
  - jeehoon.kang@kaist.ac.kr or https://cp.kaist.ac.kr/helpdesk

Thank you!